Blatt 02: Git-Quest & Calculator (Git Basics, Lambda-Ausdrücke, Gradle)

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Git Status erklären

Betrachten Sie die folgende Ausgabe von `git status` in einer lokalen Workingcopy (*Arbeitskopie*):

```
pm-lecture % git status
On branch b03

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   CONTRIBUTING.md
        modified:   homework/b03.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        foo.java

no changes added to commit (use "git add" and/or "git commit -a")
```

Erklären Sie die Ausgabe.

Gradle

Folgen Sie der Anleitung auf gradle.org und installieren Sie Gradle auf Ihrem Rechner. Legen Sie in der Konsole ein neues Gradle-Projekt für eine Java-Applikation an (ohne IDE!). Das Build-Script soll in Groovy erzeugt und als Test-API soll JUnit4 verwendet werden.

Wie finden Sie auf der Konsole heraus, welche Tasks es gibt? Erklären Sie das Projektlayout, d.h. wo kommen beispielsweise die Java-Dateien hin?

Erklären Sie, in welche Abschnitte das generierte Buildskript unterteilt ist und welche Aufgaben diese Abschnitte jeweils erfüllen. Gehen Sie dabei im *Detail* auf das Plugin `application` und die dort bereitgestellten Tasks und deren Abhängigkeiten untereinander ein.

Öffnen Sie das Projekt in Ihrer IDE. Wie können Sie hier die verschiedenen Tasks ansteuern?

Machen Sie sich Notizen, welche Sie im Praktikum nutzen dürfen, um dort das Buildskript zu erklären.

Calculator: Anonyme Klassen und Lambda-Ausdrücke

Klonen Sie die Vorgaben “Calculator” und laden Sie das Projekt als Gradle-Projekt in Ihre IDE.

In den Vorgaben wird das Package `calculator` verwendet. Recherchieren Sie und erklären Sie im Praktikum, was Packages in Java sind und wie man damit die Code-Basis strukturieren kann. Gehen Sie dabei auch auf Sichtbarkeiten ein.

Im Package `calculator` finden Sie einige Interfaces und Klassen, mit denen man einen einfachen Taschenrechner modellieren kann: Dieser kann einfache mathematische Operationen auf zwei Integern ausführen.

In der Klasse `calculator.Calculator` finden Sie vier mit `TODO` markierte Stellen in der Methode `setupOperationSelector`:

1. Erstellen Sie eine neue **Java-Klasse** `Sub`, die das Interface `Operation` implementiert und eine Subtraktion bereitstellt. Erweitern Sie den `Calculator` und binden Sie eine **Instanz dieser Klasse** ein. Nutzen Sie hier keine anonymen Klassen oder Lambda-Ausdrücke.
2. Erstellen Sie eine weitere Operation “Mul” (Multiplikation von zwei Integern). Nutzen Sie dazu eine passende **anonyme Klasse**.
3. Erstellen Sie eine weitere Operation “Div” (Integerdivision). Erstellen Sie einen passenden **Lambda-Ausdruck**.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *c1867fd (tooling: remove numbering of homework tasks, 2025-04-23)*